ЦЕНТРАЛЬНЫЙ УНИВЕРСИТЕТ

QUIC
Потоки, TLS и смена сети



Разработка
Сетевые технологии

04

QUIC

Транспорт для нескольких обменов

Сетевые технологии · лекция 4

Наше место в модели OSI

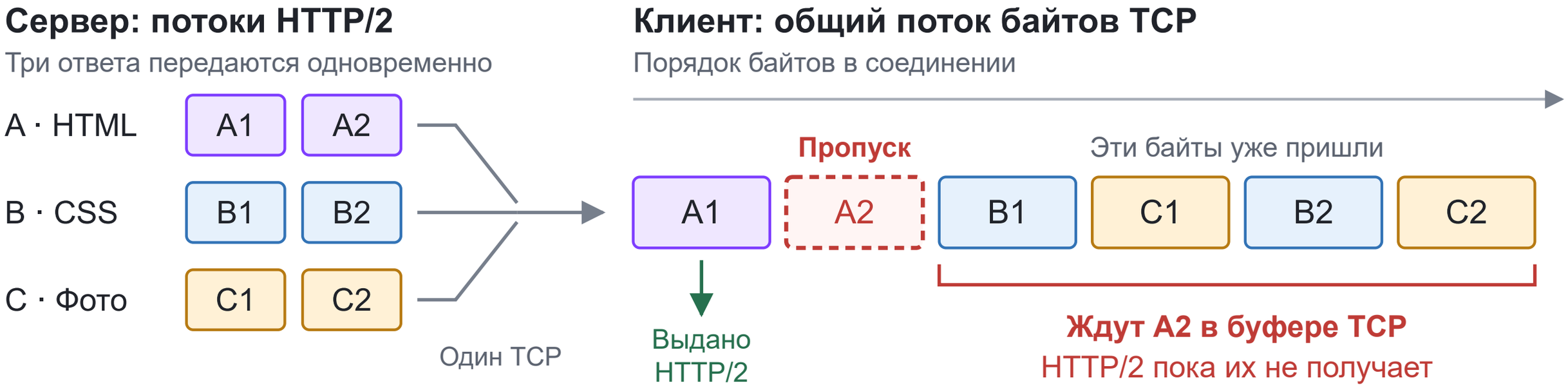
OSI — Open Systems Interconnection: карта функций сетевого взаимодействия.

Уровень	Функция	В курсе
L7	Прикладной	Лекция 1: HTTP и DNS
L6	Представления	Лекция 2: данные и TLS
L5	Сеансовый	Лекция 3: сеанс и соединение
L4	**Транспортный**	**Сегодня: QUIC поверх UDP**
L3	Сетевой	Доставка между узлами
L2	Канальный	Передача в канале
L1	Физический	Сигналы

> Границы условны: один протокол может выполнять функции нескольких уровней.

HTTP/2: потеря в TCP задерживает ответы

Браузер запросил страницу, стили и изображение. HTTP/2 передаёт все ответы через одно TCP-соединение.



A1…C2 — условные части ответов, а не границы TCP-пакетов. TLS на схеме опущен.

HOL — Head-of-Line blocking: TCP не выдаёт байты за пропуском, даже если они относятся к другим ответам.

> QUIC позволяет доставлять данные разных потоков независимо. HTTP/3 использует эту возможность для запросов и ответов.

HTTP/3 использует транспорт QUIC

В HTTP/3 за доставку независимых потоков отвечает QUIC. Сравним его стек с HTTP/2.

HTTP/2
TLS: защита записей
TCP
IP

HTTP/2 поверх TCP

HTTP/3
QUIC + TLS 1.3
UDP
IP

HTTP/3 поверх QUIC

В QUIC протокол TLS 1.3 согласует ключи, а сам QUIC защищает пакеты.

TLS — Transport Layer Security; UDP — User Datagram Protocol; IP — Internet Protocol.

> HTTP/3 задаёт формат запросов и ответов. QUIC доставляет их данные и может обслуживать другие протоколы, например DNS.

Транспортные функции поверх UDP

UDP передаёт датаграммы без гарантии доставки. Надёжность и защиту добавляет QUIC на клиенте и сервере.

Задача	Роль QUIC
Поддерживать соединение	Хранить ключи, потоки и сведения о доставке
Доставить данные потоков	Подтверждать приём и повторять потерянные данные
Защитить обмен	TLS 1.3 и защита QUIC-пакетов
Не переполнить получателя и путь	Лимиты данных и управление перегрузкой

По функциям QUIC похож на **SCTP со встроенной защитой на основе TLS 1.3**. Защищается само QUIC-соединение; IPsec может защищать IP-трафик разных приложений.

> В одном QUIC-соединении приложение открывает несколько потоков. Посмотрим, как они устроены.

У каждого потока свой порядок байтов

Поток (stream) — последовательность байтов внутри QUIC-соединения. Для потоков A и B QUIC отдельно отслеживает доставку.

Одно QUIC-соединение · смещения байтов отдельные для каждого потока



Ждёт байты 100–199
200–299 пока в буфере

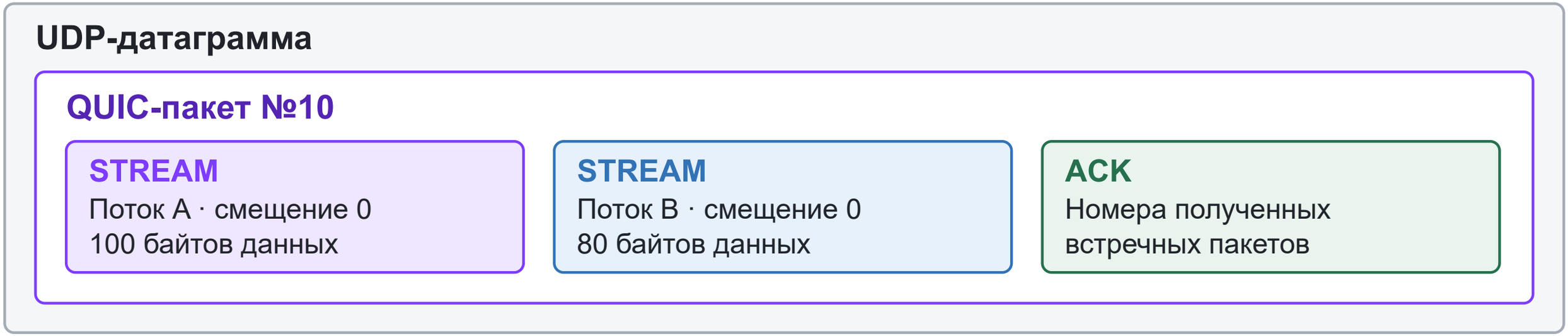
Байты 0–299 доступны
приложению без ожидания A

Вид потока	Кто передаёт данные
Двунаправленный	Обе стороны, со своими смещениями
Однонаправленный	Только сторона, открывшая поток

> QUIC передаёт поток байтов, как TCP: границы сообщений задаёт приложение. SCTP, напротив, сохраняет границы сообщений.

Поток, фрейм, пакет, датаграмма

QUIC помещает части потоков в STREAM-фреймы, а фреймы собирает в пакеты. Один пакет может нести данные разных потоков.



ACK — Acknowledgment: подтверждение получения пакетов.

> Номер пакета нужен для подтверждения доставки. Смещение показывает, куда поместить байты в потоке.

Какие байты QUIC уже выдаст приложению?

Один получатель, два потока. [0,100) означает 100 байтов со смещения 0.

Пакет	Данные	Событие
10	A[0,100)	Пришёл
11	A[100,200)	Потерян
12	B[0,100)	Пришёл
13	A[200,300)	Пришёл

Что изменится, если пакет 11 содержал также B[100,200)?

A выдаёт [0,100), B выдаёт [0,100). Если в 11 были байты B, у B тоже есть пропуск.

Что может задержать сразу оба потока

Что связывает A и B	Возможное следствие
Один потерянный пакет несёт оба потока	У обоих возникает пропуск
Общее окно перегрузки	После потери отправитель может замедлить оба потока
Лимит всего соединения	Отправитель ждёт увеличения лимита получателем
Зависимость приложения	Байты B уже есть, но использовать их пока нельзя

QUIC устраняет общее ожидание пропуска, знакомое по TCP. Но потоки всё ещё делят сеть и ресурсы получателя.

Вернёмся к DNS, как он может работать?

Запрос тот же: узнать IP по имени. Меняется канал между DNS-клиентом и резолвером.

Вариант	Стек и стандартный порт	Где поддерживается
DNS	DNS → UDP или TCP · **53**	Повсеместно; без шифрования
DoT	DNS → TLS → TCP · **853**	ОС и DNS-прокси; широко
DoH / HTTP/2	DNS → HTTP/2 → TLS → TCP · **443**	Браузеры, ОС, прокси; широко
DoH / HTTP/3	DNS → HTTP/3 → QUIC → UDP · **443**	Нужна поддержка HTTP/3 у DNS-клиента и сервера
DoQ	DNS → QUIC → UDP · **853**	Специализированные клиенты; AdGuard DNS, Quad9

> DoQ и DoH — разные протоколы. В DoQ нет HTTP; DoH через HTTP/3 — вариант DoH.

DoH — DNS over HTTPS · DoT — DNS over TLS · DoQ — DNS over QUIC

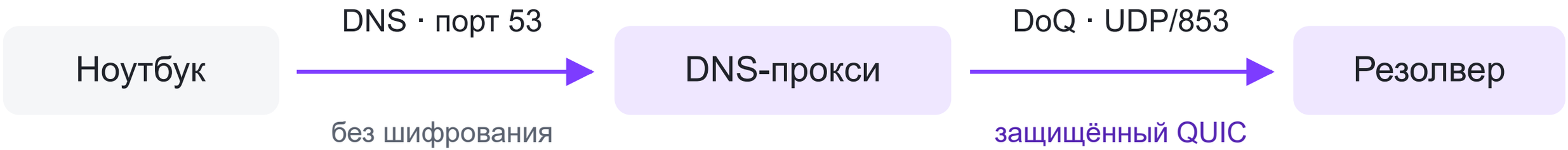
Где включить защищённый DNS?

Где настраиваем	Что указываем	Какой канал получаем
Chrome / Firefox	Защищённый DNS: провайдер или HTTPS URL	DoH для запросов браузера; HTTP/3 зависит от реализации
Android · Private DNS	Имя DNS-провайдера	DoT; для некоторых провайдеров возможен DoH3
Windows 11	DNS-сервер и настройка DoH	Системный DoH
iOS / macOS	DNS-профиль или приложение	Встроенные DoH / DoT
DNS-прокси / роутер	Например, AdGuard Home: `quic://dns.adguard-dns.com`	DoQ от прокси к резолверу

Не любой роутер понимает DoQ. Нужен DNS-клиент с поддержкой выбранного протокола.

> Уже первый запрос адреса сайта может идти защищённо. Сайт не включает DoQ для следующих доменов.

Проверяем каждый участок пути DNS



Браузеру не нужна поддержка DoQ: его DNS-запросы пересылает прокси.

Другой вариант	Что меняется	Поддержка
DNS внутри VPN	DNS/53 защищён туннелем до VPN-шлюза	Распространён; дальше может быть открытый DNS
DNSCrypt	Собственная защита DNS поверх UDP/TCP	Специальный клиент, например dnscrypt-proxy
ODoH	HTTPS-реле отделяет IP клиента от DNS-запроса	Ограниченная; нужны клиент, реле и резолвер

> DNS внутри VPN ≠ DoQ. Проверяйте, какие участки пути действительно зашифрованы.

QUIC используют не только для веба

Новый прикладной протокол может использовать QUIC напрямую или работать через HTTP/3. Менять сам QUIC для этого не обязательно.

Пример	Задача	Как использует QUIC
SMB over QUIC	Доступ к сетевым файлам	Напрямую; реализован в Windows
MQTT over QUIC	Сообщения и телеметрия	Напрямую; реализация EMQX
Media over QUIC (MoQ)	Доставка аудио и видео	Напрямую или через WebTransport
WebTransport	Потоки и датаграммы для веб-приложений	Вариант поверх HTTP/3

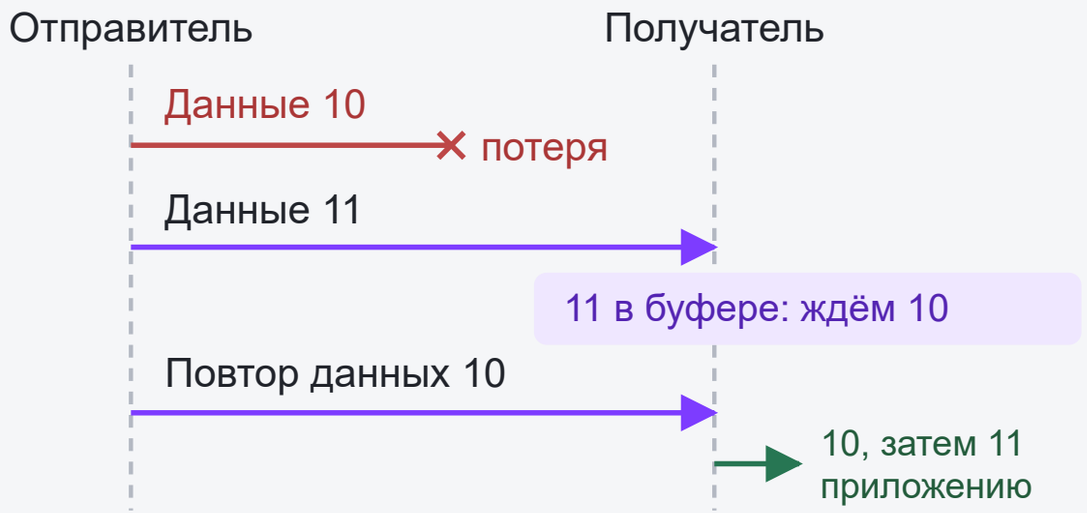
MoQ и WebTransport over HTTP/3 пока описаны в проектах стандартов IETF.

> QUIC отвечает за транспорт. Смысл данных — файлы, сообщения или медиаконтент — задаёт прикладной протокол.

QUIC DATAGRAM: новое состояние важнее старого

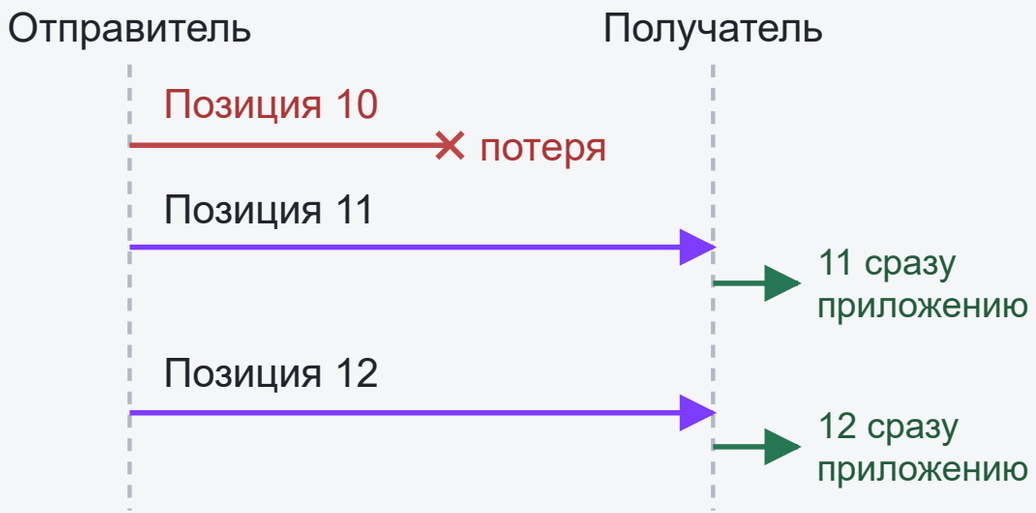
Игра отправляет полное положение игрока каждые 50 мс. Обновление №10 потерялось, но №11 уже пришло: ждать старую позицию не нужно.

STREAM · один поток байтов



Пропуск задерживает следующие байты

DATAGRAM · отдельные сообщения



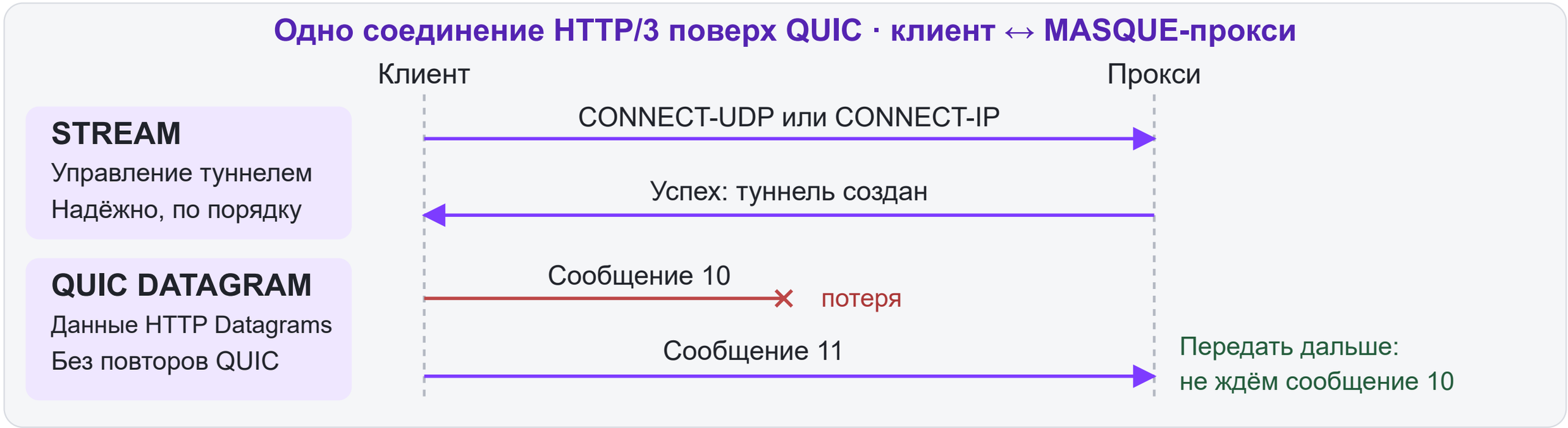
Потерянное сообщение не повторяется QUIC

> В одном QUIC-соединении: важные команды — в потоках, частые обновления состояния — в DATAGRAM.

Шифрование и контроль перегрузки остаются. Поддержку DATAGRAM стороны согласуют заранее.

MASQUE: управление и пакеты идут разными путями

MASQUE расширяет HTTP: **CONNECT-UDP** создаёт UDP-прокси, **CONNECT-IP** — IP-туннель. Ниже показан вариант HTTP/3 с QUIC DATAGRAM.

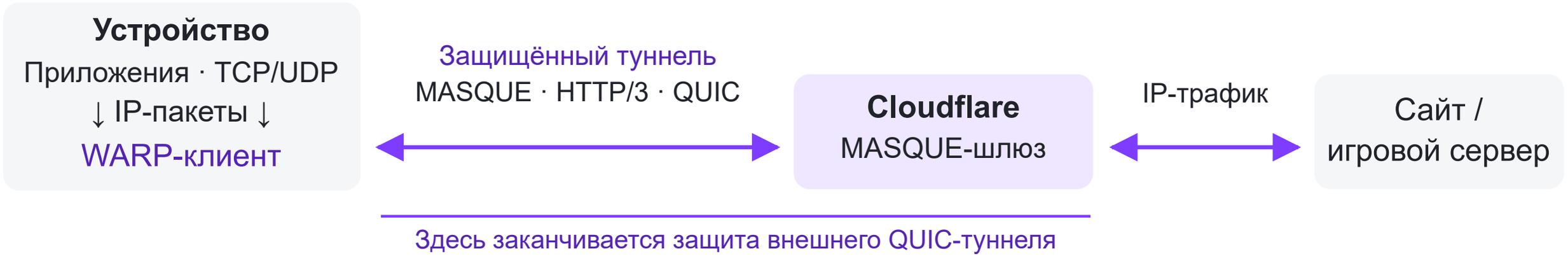


HTTP Datagram связан с запросом CONNECT, но не идёт внутри его STREAM. Потерянный DATAGRAM не повторяется QUIC.

Если данные передавать в Capsules через надёжный поток, ожидание пропуска вернётся: в этом потоке, а при HTTP/2 — во внешнем TCP.

Cloudflare WARP: MASQUE на практике

WARP — клиент и сервис Cloudflare. В режиме туннелирования он забирает IP-пакеты приложений и передаёт их в сеть Cloudflare через MASQUE.



Сценарий	Что делает WARP
Обычные TCP/UDP-соединения	Направляет пакеты в туннель; приложению не нужна поддержка MASQUE
Обычный сервер назначения	Cloudflare выпускает трафик из туннеля; серверу тоже не нужен MASQUE
Режим DNS only	Защищает только DNS; это не туннель для трафика приложений

> WARP защищает путь до Cloudflare. HTTPS отдельно защищает данные приложения до сайта.

Как TLS 1.3 защищает соединение QUIC

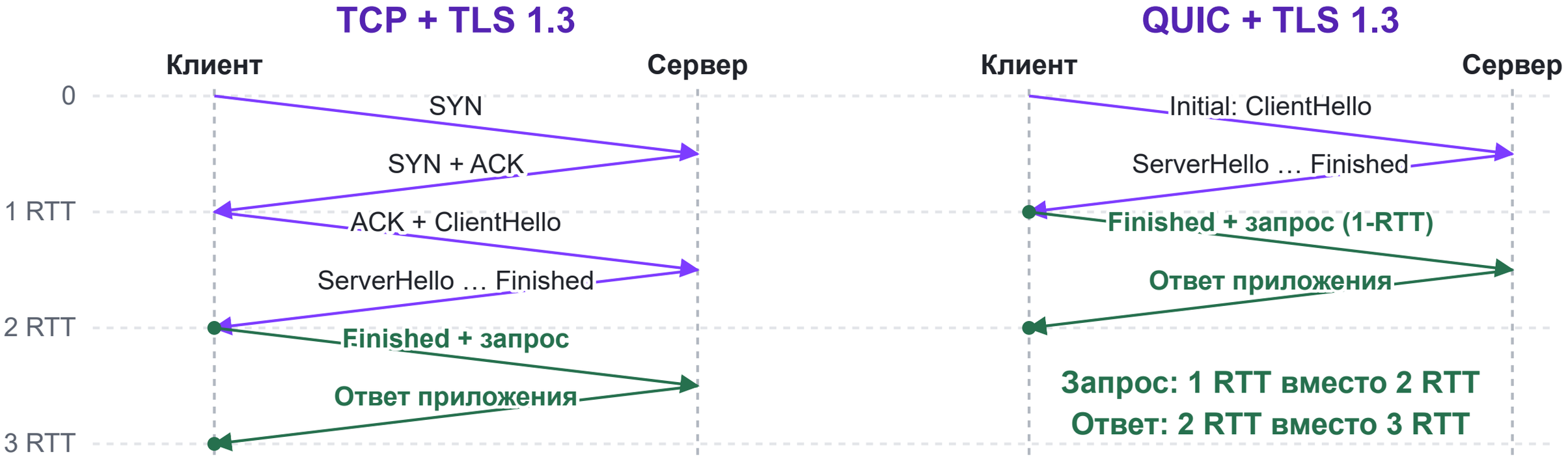
До обмена данными клиент и сервер согласуют ключи с помощью TLS 1.3. Этот обмен называют handshake.

Участник механизма	Что делает
TLS 1.3 handshake	Согласует ключи и проверяет подлинность сервера
CRYPTO-фреймы QUIC	Переносят сообщения handshake
Защита пакетов QUIC	Защищает содержимое пакетов
STREAM-фреймы QUIC	Переносят байты приложения

> QUIC сам переносит сообщения TLS и защищает пакеты. Отдельный слой записей TLS ему не нужен.

Когда клиент отправит первый запрос?

Новое соединение, адрес сервера известен. RTT — время обмена туда и обратно.

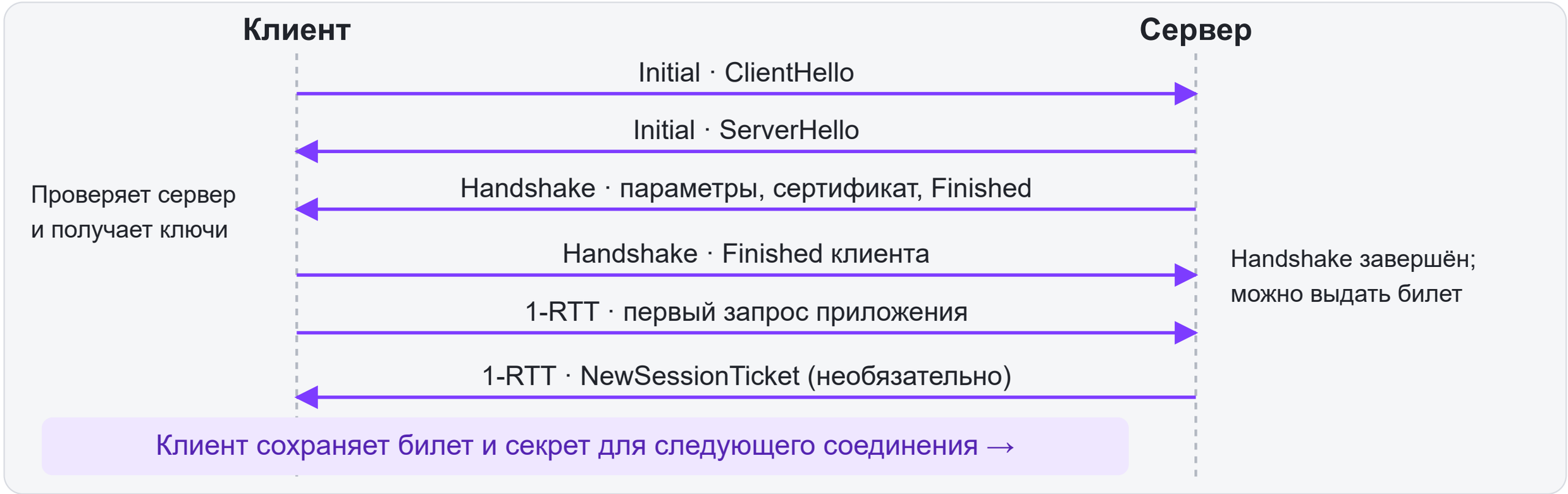


Время идёт сверху вниз. Стрелка занимает ½ RTT; обработка мгновенна.

Без потерь, Retry, HelloRetryRequest, TCP Fast Open и 0-RTT.

> QUIC совмещает установление транспорта и TLS: запрос уходит на один RTT раньше.

Установление нового соединения QUIC

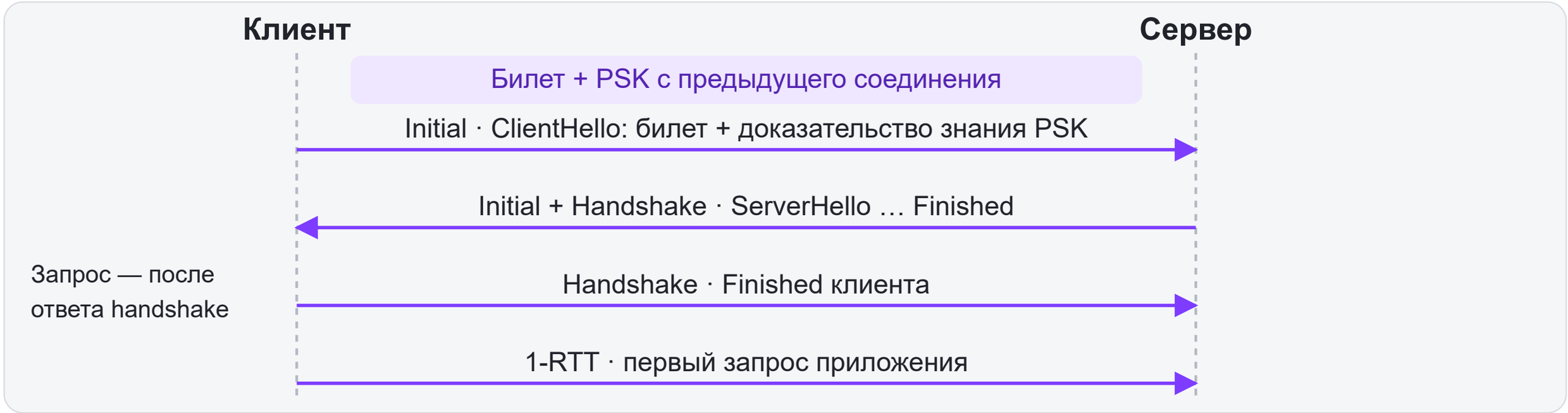


Время идёт вниз. TLS-сообщения идут в CRYPTO-фреймах; запрос — в STREAM. Finished и запрос находятся в отдельных пакетах.

> Запрос не ждёт билета. NewSessionTicket нужен для следующего соединения.

Возобновление: без 0-RTT

Предыдущее соединение закрыто. Клиент снова подключается к тому же серверу — уже с сохранённым билетом.

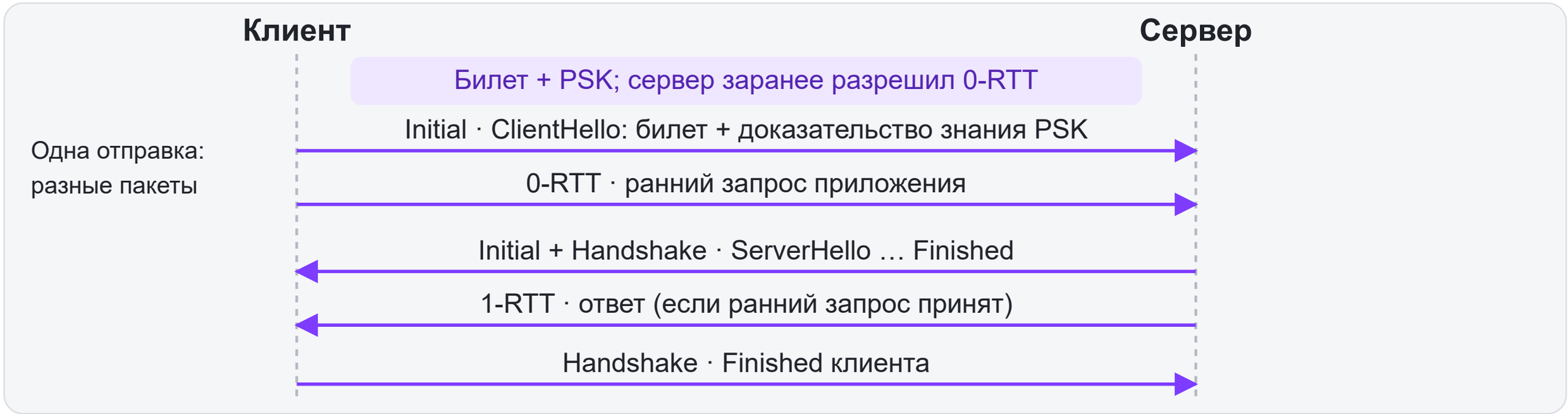


PSK — Pre-Shared Key, сохранённый секрет возобновления. Клиент доказывает знание секрета, не передавая сам секрет.

Билет сокращает TLS-обмен, но запрос всё ещё ждёт 1 RTT. На следующем слайде уберём это ожидание.

Возобновление: с 0-RTT

Клиент снова подключается с билетом. Если сервер заранее разрешил ранние данные, запрос можно отправить сразу.



Initial и 0-RTT — разные QUIC-пакеты; они могут идти в одной UDP-датаграмме. Сервер вправе отклонить ранние данные.

> Запрос не ждёт handshake. Но 0-RTT допускает replay: повтор записанного запроса может повторить действие.

Разбор: какой запрос можно отправить рано?

Операция	Что сделает повтор?	Для 0-RTT
Прочитать публичный каталог без побочных эффектов	Повторное чтение и нагрузка	✅ Можно*
Создать заказ	Может появиться ещё один заказ	❌ Нельзя
GET, который списывает бонус	Может произойти повторное списание	❌ Нельзя

Какие операции безопасны при повторе запроса?

* Если сервер разрешает 0-RTT для каталога и допускает повторную обработку. Название HTTP-метода само по себе не гарантирует безопасность.

Варианты установления соединения

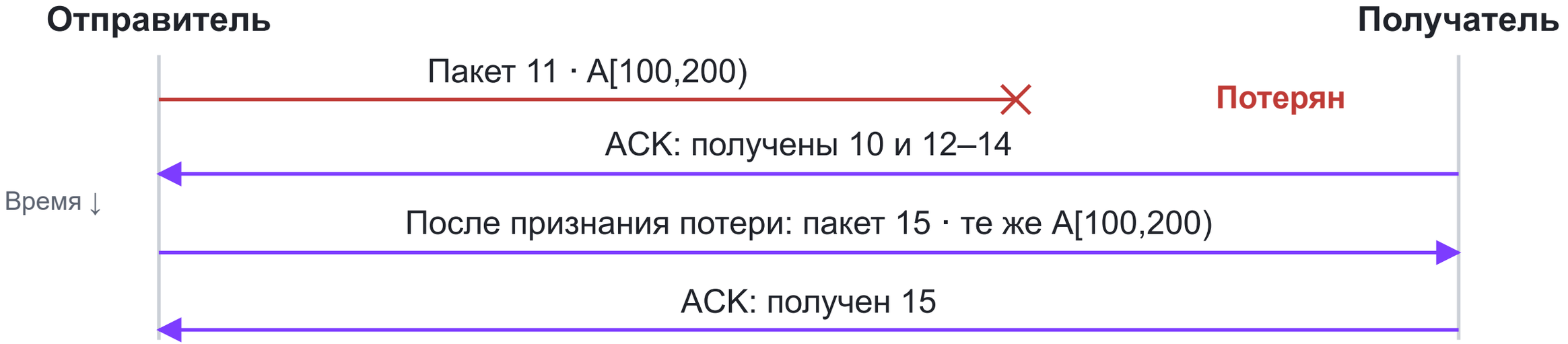
Состояние	Что происходит
Первое соединение с сервером	Выполняем полный handshake
Возобновление без ранних данных	Используем билет, но запрос ждёт handshake
Принятый 0-RTT	Запрос отправлен раньше ответа handshake
Отклонённый 0-RTT	Handshake может завершиться; запрос передаём позднее

HTTP **425 Too Early** означает отказ обрабатывать ранний запрос: клиент должен повторить его без ранних данных.

> После установления соединения QUIC следит за доставкой данных. Разберём подтверждения, потери и ограничения отправки.

Повторная отправка потерянных данных

Вернёмся к пропуску в потоке A. Когда QUIC признает пакет потерянным, он отправит нужные байты в новом пакете.



ACK перечисляет полученные пакеты диапазонами номеров. При повторной отправке смещения байтов в потоке остаются прежними.

> Подтверждение пакета не доказывает выполнение запроса приложением.

Как отправитель замечает потерю

Наблюдение	Механизм
Подтверждены пакеты с более поздними номерами	QUIC оценивает отставание по номеру пакета и времени с учётом RTT
Подтверждения долго не приходят	По таймеру PTO (Probe Timeout) отправитель посылает пробные пакеты

Пакеты могут прийти в другом порядке. Поэтому один пропущенный номер в ACK ещё не доказывает потерю.

> Порог потери позволяет начать восстановление данных. Таймер PTO помогает получить обратную связь и сам по себе не объявляет пакеты потерянными.

Получатель ограничивает новые данные

Управление потоком (flow control): получатель задаёт лимиты данных для каждого потока и всего соединения.

Лимит	Разрешено	Учтено	Остаток
Поток A	1000 Б	900 Б	100 Б
Соединение	5000 Б	4950 Б	50 Б

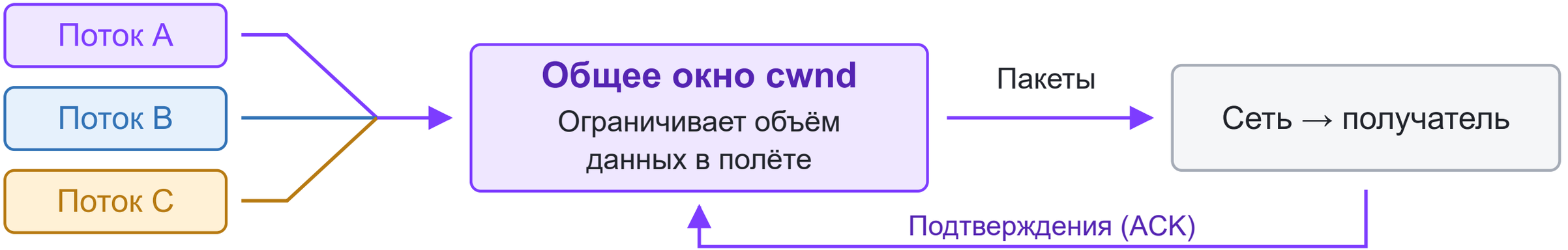
Сеть позволяет отправлять. Сколько новых байтов A разрешает получатель?

Не более 50 Б. Увеличение только лимита потока A не поможет.

MAX_STREAM_DATA — предел потока; MAX_DATA — предел соединения.

Потоки делят общее окно перегрузки

Получатель готов, но сеть может быть перегружена. Отправитель ограничивает объём данных в полёте с помощью **cwnd**, как в TCP.



Управление перегрузкой (congestion control) учитывает доставку пакетов. После потери окно cwnd может уменьшиться для всех потоков.

Pacing распределяет отправку пакетов во времени, чтобы не посылать их одним всплеском.

> Байты B можно выдать независимо от A, но скорость обоих потоков ограничивает общее окно перегрузки.

Почему отправитель или приложение ждёт

Причина ожидания	Что проверить
В потоке A не хватает байтов	Какие смещения получены и что отправлено повторно
Получатель не разрешает новые данные	Остаток лимита потока и всего соединения
Заполнено окно перегрузки	cwnd, данные в полёте и подтверждения
Байты получены, приложение ждёт	Какие ещё данные нужны для обработки

До сих пор путь не менялся. Теперь телефон переключается с Wi-Fi на мобильную сеть: как сохранить соединение?

Идентификатор соединения и адрес клиента

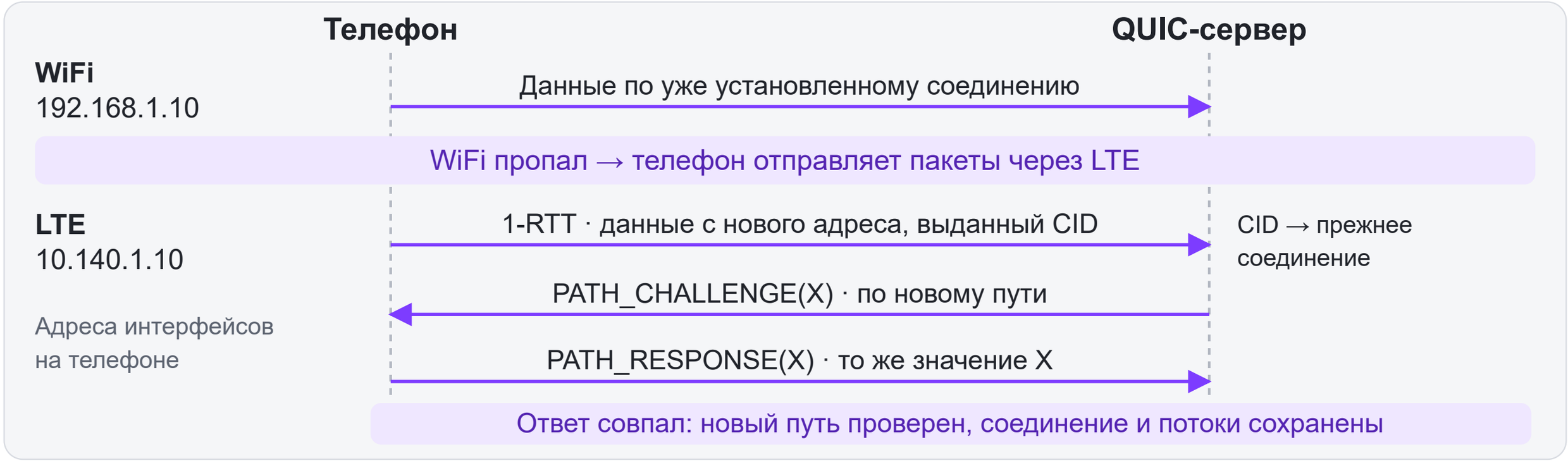
При смене сети могут измениться адрес и порт клиента. **CID — Connection ID** позволяет серверу связать пакет с уже существующим соединением.

До смены сети	После смены сети
Адрес X, UDP-порт p	Адрес Y, UDP-порт q
Ключи и потоки соединения	Могут сохраниться прежними
Текущий CID	Может смениться на другой выданный CID

> Адрес и порт нужны для доставки пакета. CID помогает найти соединение, к которому этот пакет относится.

Новый путь требует проверки

Телефон меняет сеть. Handshake уже подтверждён, миграция разрешена, сервер заранее выдал CID.



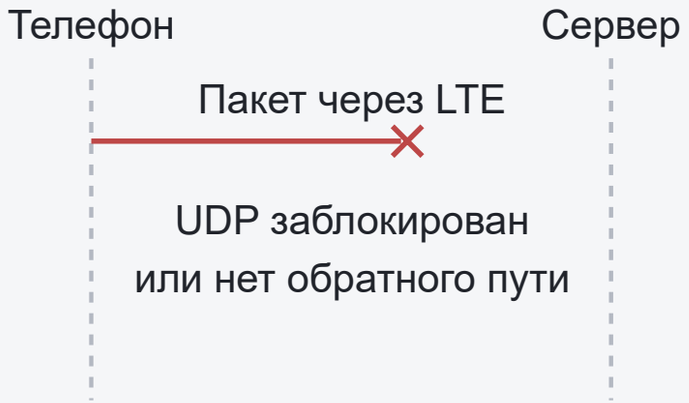
Это локальные IP телефона. При NAT сервер видит внешние IP и UDP-порты WiFi-роутера и оператора, а не эти адреса.

> CID помогает найти соединение. Ответ на PATH_CHALLENGE подтверждает доступность нового пути в обе стороны.

Когда смена сети прерывает обмен

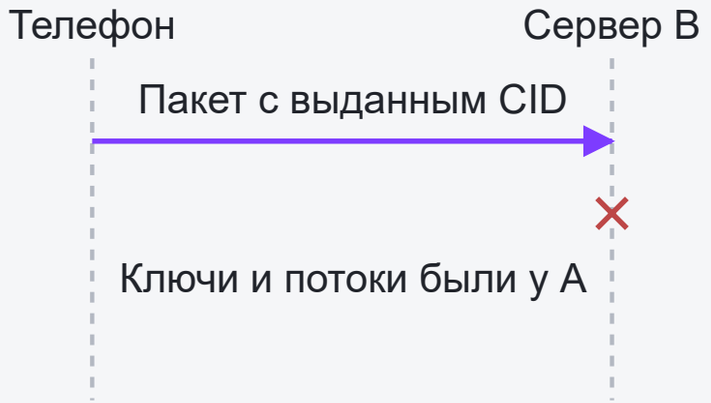
До перехода: миграция должна быть разрешена, а подходящие CID — доступны

1. Новый путь недоступен



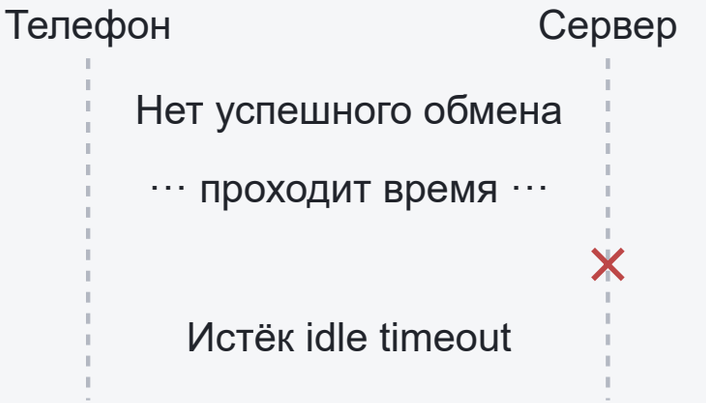
PATH_RESPONSE не приходит.
Новый путь не подтверждён.

2. Нет состояния соединения



B не может продолжить обмен.
CID не переносит состояние.

3. Переключение затянулось



Большой RTT и потери → пауза.
Долгая пауза → закрытие.

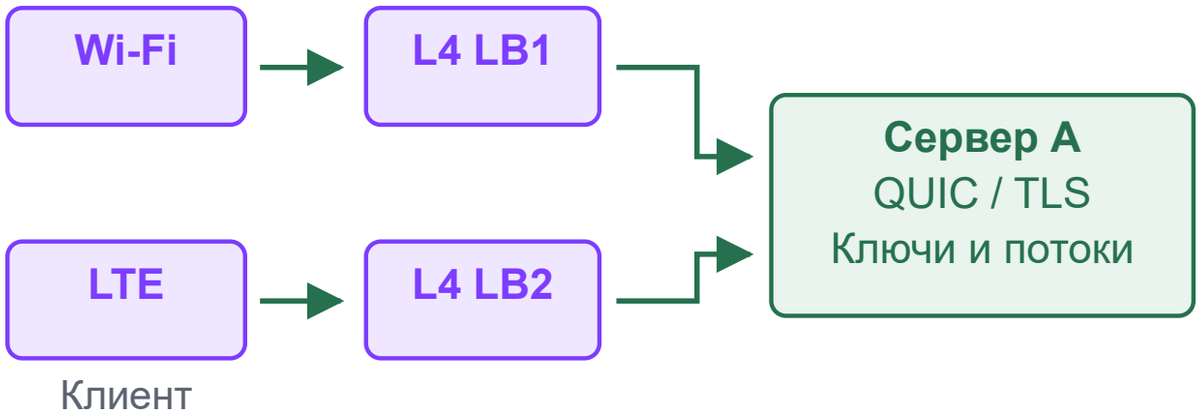
Базовый QUIC v1 не обещает одновременную передачу по нескольким путям.

> Нового доступного пути недостаточно: балансировщик должен доставить пакет серверу, который хранит соединение.

L4 и L7: где живёт QUIC-соединение

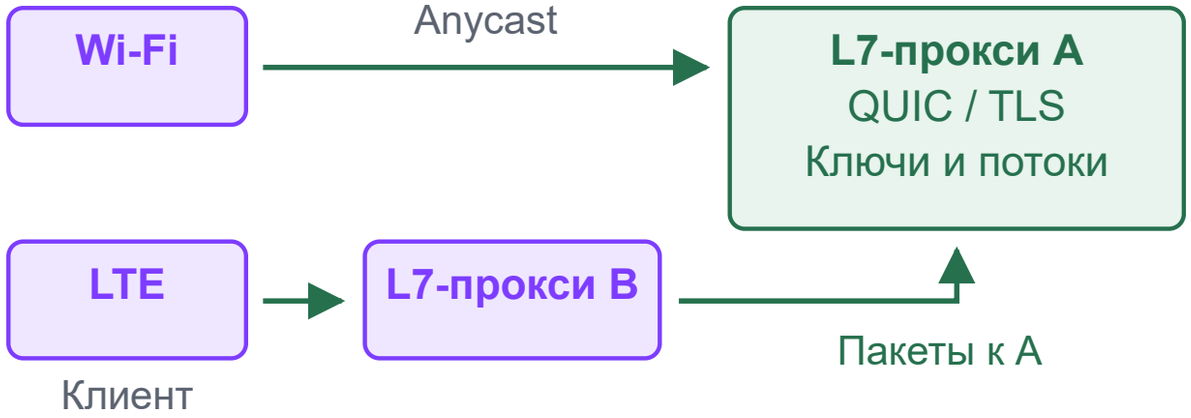
При переходе с Wi-Fi на LTE пакет может попасть на другой балансировщик. Где завершается QUIC/TLS и хранится состояние?

L4: пересылка пакетов



LB1 и LB2 маршрутизируют по CID к одному владельцу соединения.

L7 + Anycast: терминация на прокси



B пересылает пакеты владельцу A или принимает его полное состояние.

Миграция QUIC сохраняет соединение, только если инфраструктура сохраняет доступ к его состоянию. Попадание на другой L7-прокси само по себе не обеспечивает бесшовность.

HTTP/3 сохраняет смысл HTTP

Методы, адреса ресурсов и коды ответа остаются знакомыми. Меняются формат передачи и транспорт.

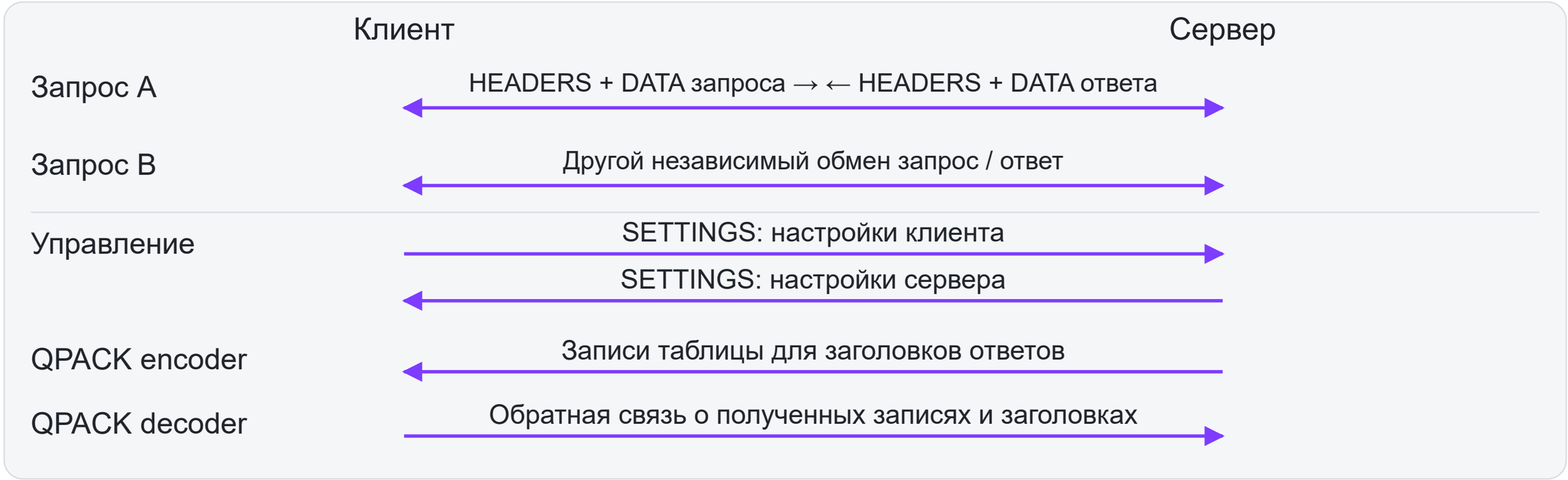
Что передаём	Как это устроено в HTTP/3
Метод, ресурс, статус	Поля запроса и ответа
Тело сообщения	DATA-фреймы HTTP/3
Отдельный запрос/ответ	Двунаправленный QUIC-поток

ALPN — Application-Layer Protocol Negotiation: выбор прикладного протокола в TLS. Значение `h3` означает HTTP/3.

> Версии HTTP, QUIC и TLS согласуются по разным правилам.

Запросы и служебные потоки HTTP/3

Запросы идут по двунаправленным потокам, служебные данные — по однонаправленным.

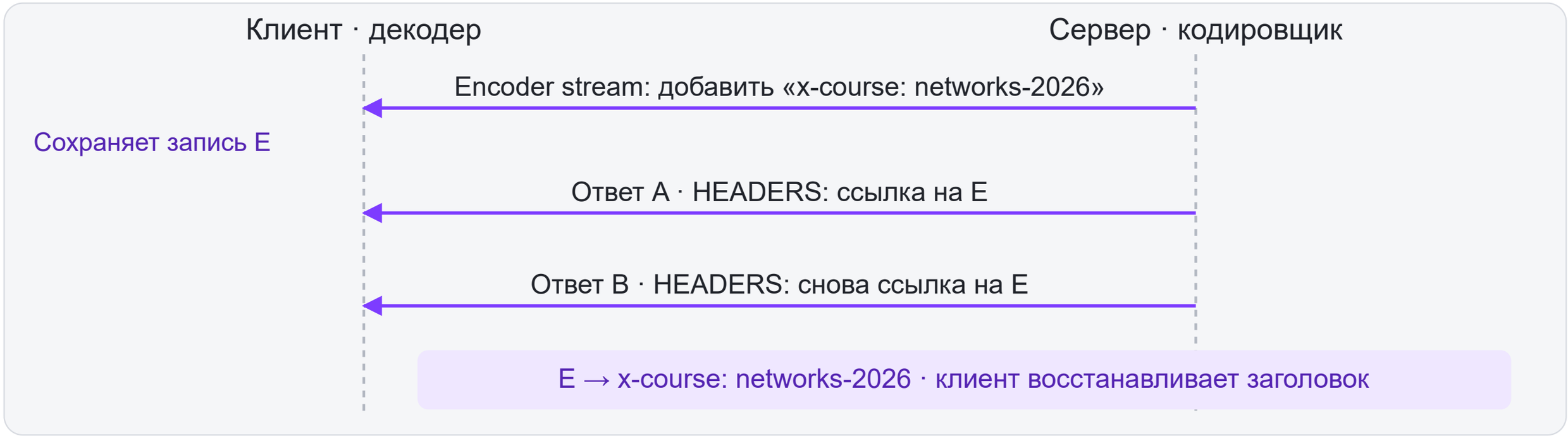


Каждая служебная стрелка — отдельный однонаправленный поток. Для QPACK показано сжатие ответов; для запросов роли зеркальны.

> HEADERS и DATA — фреймы HTTP/3 внутри потока. QUIC переносит его байты в STREAM-фреймах.

QPACK: вместо заголовка — ссылка на запись

Сервер повторяет `x-course: networks-2026` в ответах. QPACK позволяет передать эту пару один раз, затем использовать ссылки.

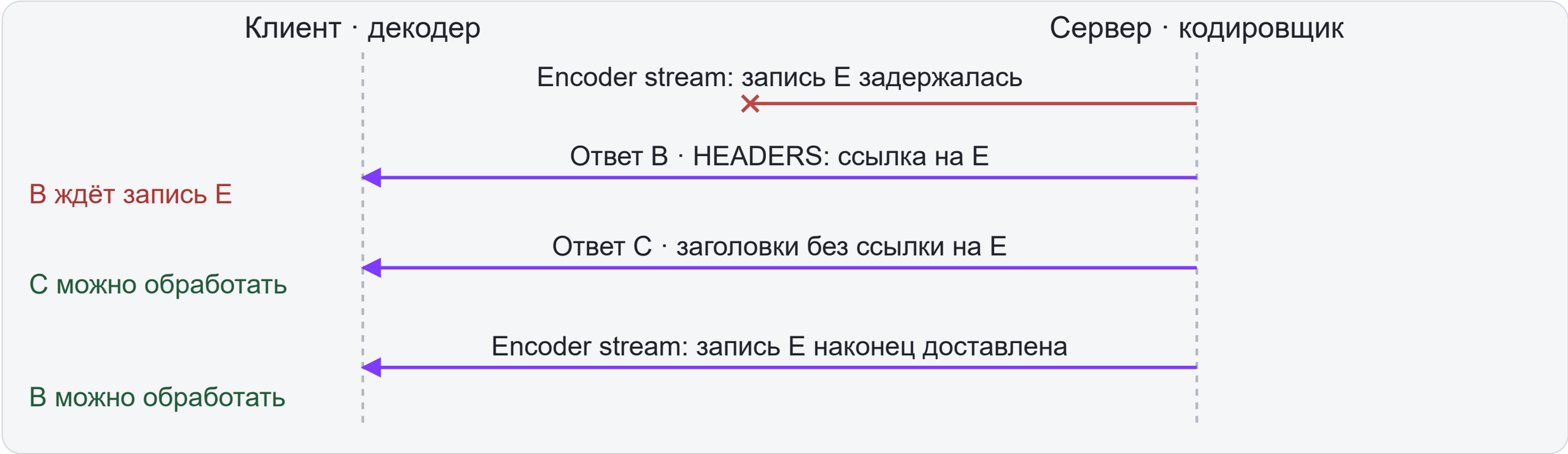


E — условное имя записи динамической таблицы. Таблица общая для ответов в этом соединении; тело ответа она не сжимает.

> Экономим байты заголовков. Цена: чтобы раскрыть ссылку, клиент должен получить соответствующую запись.

QPACK: ссылка пришла раньше записи

Теперь запись E задержалась, а HEADERS ответа B уже доставлен. QUIC разрешает такой порядок между разными потоками.



Избежать ожидания можно, передав значение целиком или сославшись на запись, получение которой уже подтверждено.

> QUIC доставил байты B, но HTTP/3 ещё не может прочитать заголовки. Ждут ответы, которым нужна недоставленная запись.

Как клиент узнаёт о HTTP/3

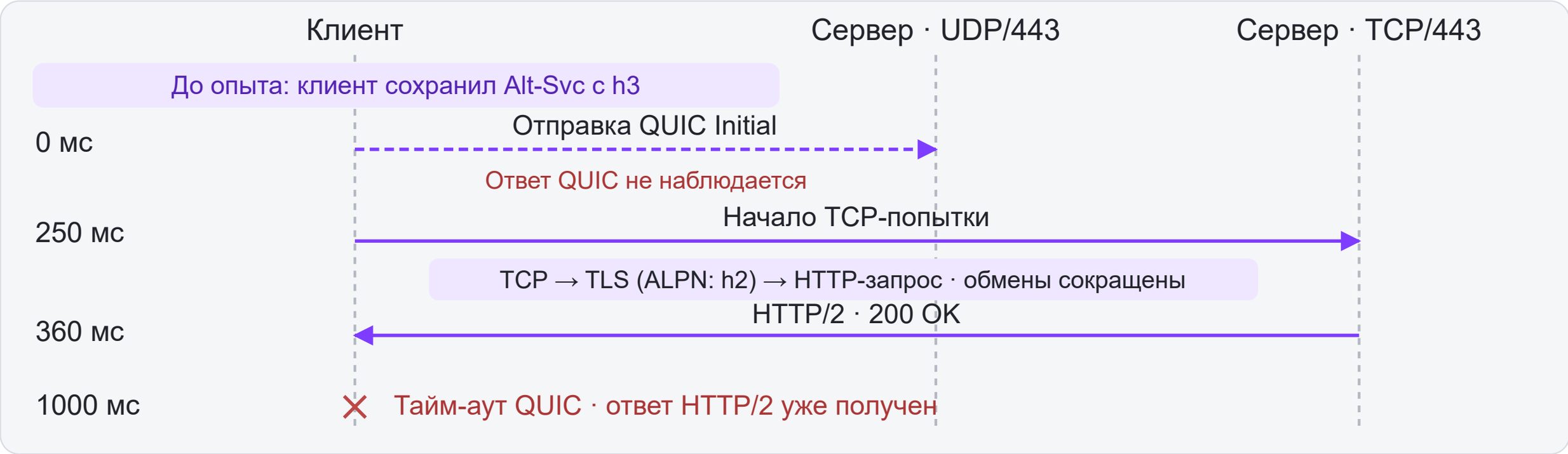
Источник	Пример или смысл
HTTP-ответ, например по HTTP/2	`Alt-Svc: h3=":443"`
DNS-запись HTTPS	`example.test. IN HTTPS 1 . alpn="h3,h2"`
Кеш клиента	Сохранённое объявление поддержки HTTP/3

Alt-Svc — Alternative Services, альтернативный сервис.
HTTPS здесь также название типа DNS-записи.

> Узнав о поддержке HTTP/3, клиент может попробовать QUIC. Успех зависит от доступности сервера и сети.

Сервер объявил HTTP/3, клиент выбрал HTTP/2

Учебные времена одного клиента, не универсальные таймеры. Шкала времени условная.



Достаточно ли этих событий, чтобы доказать блокировку UDP?

Клиент попробовал QUIC, затем получил ответ по HTTP/2. Это fallback — переход к другому варианту. Причина неудачи QUIC пока неизвестна.

Что видно в QUIC и что скрывает ECH

SNI — имя сервера в TLS ClientHello. Посмотрим, что из него доступно наблюдателю между клиентом и сервером.

В обоих случаях видны IP, UDP-порты, размеры пакетов и время передачи

Без ECH

Клиент → Наблюдатель → Сервер

Из QUIC Initial можно прочитать:

ClientHello
SNI = private.example

Защита Initial не скрывает ClientHello.

С ECH · успешно принят сервером

Клиент → Наблюдатель → Сервер

Снаружи: ClientHelloOuter и публичное имя

Зашифрованный ClientHelloInner
Внутри: SNI = private.example

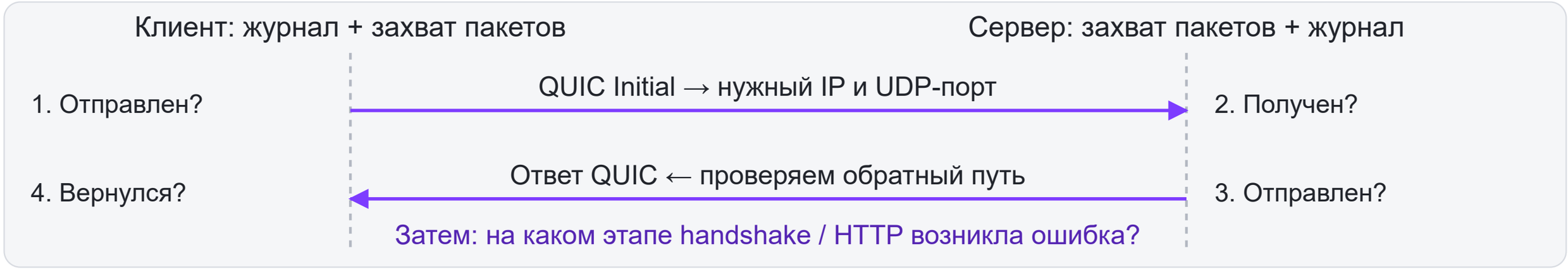
ECH-сервер расшифровывает внутреннее имя.

ECH — Encrypted Client Hello, отдельный механизм TLS: работает и с QUIC, и с TCP. Незашифрованный DNS и IP могут выдать назначение.

> ECH скрывает внутренний ClientHello, но не адрес сервера. Переход на HTTP/2 сам по себе не устраняет фильтрацию ECH или IP.

Как выяснить причину неудачного соединения

Вернёмся к fallback: HTTP/2 ответил, а QUIC — нет. Сравним одну попытку на обоих концах соединения.



Initial не виден у сервера

Проверить адрес, порт, маршрут и фильтрацию по пути.

Initial есть, ответа нет

Не отправлен → сервис / конфиг.
Отправлен → обратный путь.

Ответ пришёл, но обмен не удался

Проверить TLS, ALPN, код ошибки и этап завершения соединения.

Сопоставляем время, имя, IP и порт назначения, CID, этап и ошибку. Учитываем NAT: адрес клиента на сервере может отличаться.

> Нет ответа — это симптом, а не доказательство блокировки. HTTP/3 подтверждают согласованный ALPN h3 и журнал обмена.

Проверим понимание QUIC

- Потерян пакет с байтами A. Обязательно ли ждёт B?
- Сервер отклонил 0-RTT. Обязательно ли разорвалось соединение?
- Alt-Svc содержит h3. Обязательно ли текущий запрос пойдёт по HTTP/3?

B может продолжить обмен, если его данные доступны. Отклонение 0-RTT не мешает завершить handshake. Объявление h3 не гарантирует подключение.

Что будем разбирать на семинаре

Проследим, как QUIC ведёт себя при потерях, ограничениях получателя и смене сети.

- По событиям доставки определим, какие данные получит приложение.
- По установлению соединения оценим задержку и риск повтора запроса.
- По журналам объясним смену пути и переход с HTTP/3 на HTTP/2.

Лонгрид: схемы, справочник, задачи и ответы

> Для каждого случая объясним результат и выберем наблюдение, которым можно проверить свою версию.